SUMMER INTERNSHIP REPORT ON

SCAN Me – AI-Powered Virtual Hotel Receptionist System

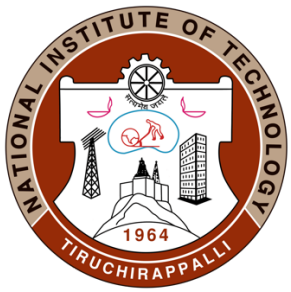
submitted in partial fulfilment of the requirements for
the completion of 7th semester of

B.Tech

in Computer Science and Engineering

By

Neeli Vishnu Vardhan (106122087)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTEOF TECHNOLOGY

TIRUCHIRAPALLI-620015

SEPTEMBER 2025

ABSTRACT

The "Scan Me – AI-Powered Virtual Hotel Receptionist System" is a full-stack web application designed to revolutionize guest interaction in the hospitality industry by introducing an intelligent, autonomous digital receptionist named Sarah for "The Grand Luxe" hotel. This project integrates cutting-edge technologies in artificial intelligence, real-time communication, and digital human avatars to deliver a seamless, engaging, and 24/7 service.

The system is built on a modular three-tier architecture that ensures scalability, security, and maintainability. The core components included:

1. Backend Layer: Built using Python and Flask, this layer serves as a secure REST API server responsible for generating LiveKit access tokens, hosting the AI agent logic, and interfacing with external services. It leverages Google's Real-Time Voice API for low-latency speech recognition and natural-sounding text-to-speech synthesis, enabling Sarah to engage in fluid and human-like conversations.
2. AI and Communication Layer: The AI agent is implemented using LiveKit Agents, which enables real-time audio/video interaction between the guest and Sarah. A key innovation is the integration of Tavus AI, a specialized platform for generating realistic, expressive digital avatars with synchronized lip movements, facial expressions, and emotional prosody. Although currently commented out in agent.py, the livekit-agents[tavus] plugin confirms its planned and functional integration, making Sarah a visually immersive and lifelike virtual receptionist.
3. Frontend Layer: The user interface was developed using React with TypeScript, styled with Tailwind CSS, and enhanced with shadcn-ui components. Built on Vite, it provides a responsive, modern, and intuitive interface with dedicated pages for hotel amenities (spa, fitness, dining), booking, and confirmations.

A defining feature of Scan Me is the implementation of the Model Context Protocol (MCP), which extends the AI agent's capabilities beyond conversation. Through a Python-based MCP client connected to an n8n automation server, Sarah can perform external actions such as:

- Checking room availability via Google Calendar API
- Booking rooms and creating calendar events
- Sending automated confirmation emails via Gmail API

- Opening relevant web pages (e.g., spa menu) in the user's browser using a custom open_url tool

However, the current implementation of the open_url tool uses Python's webbrowser.open() function, which is executed on the server side, which is a critical limitation. For proper functionality, future work must involve relaying URL commands to the frontend via LiveKit data channels, where JavaScript can execute window.open() on the user's device.

The key technical contributions are as follows:

- Real-Time Voice Interaction: Achieved through Google's Real-Time Voice API with full-duplex communication via LiveKit, enabling natural, context-aware conversations.
- Visual Avatar Integration: Implemented using Tavus AI, enhancing user engagement and trust through lifelike visual presence.
- Extended AI Capabilities: Enabled via the MCP, allowing the agent to perform real-world tasks and interact with external tools.
- Secure and Scalable Architecture: Ensured through environment variable management (python-dotenv), HTTPS enforcement, token scoping, and modular design.
- User-Centric Design: Focused on a clean, responsive, and accessible interface using React, Tailwind CSS, and shadcn-ui with intuitive navigation and real-time feedback.

The development followed a structured Software Development Life Cycle (SDLC), including requirement analysis, system design, implementation, integration, testing, and deployment. The backend was deployed on Render and the frontend on Vercel, with SSL enforcement and domain linking for public accessibility.

ACKNOWLEDGEMENTS

I express my profound gratitude to Dr. S.R. Balasundaram, Professor, Department of Computer Applications, National Institute of Technology, Tiruchirappalli, for his exceptional guidance, continuous encouragement, and invaluable mentorship throughout my summer internship. His deep technical insights, constructive feedback, and unwavering support played pivotal roles in shaping the direction and success of this project.

His ability to simplify complex concepts in artificial intelligence, real-time communication, and web application architecture provided me with a solid conceptual foundation. Regular discussions and critical reviews of the system design and implementation significantly enhanced the quality of this work and broadened my understanding of modern full-stack development practices.

I am also deeply thankful to the open-source communities and technology providers—LiveKit, Google AI, n8n, and React—whose robust and well-documented platforms enabled the realization of this advanced AI-driven application. Their tools and APIs were instrumental in integrating real-time voice, visual avatars, and intelligent agent behaviours.

I acknowledge the Department of Computer Science and Engineering, NIT Tiruchirappalli, for providing the necessary academic environment, computing resources, and intellectual stimulation that fostered innovation and learning in this study. Working on a project at the intersection of AI, real-time systems, and user experience was both challenging and immensely rewarding.

This experience not only strengthened my technical skills but also instilled in me a deeper appreciation for structured software development, user-centric design, and the transformative potential of AI in the service industry.

Sincerely,

Neeli Vishnu Vardhan

TABLE OF CONTENTS

Table of Contents

LIST OF TABLES

LIST OF FIGURES

Chapter 1 Introduction

1.1 General

In the rapidly evolving landscape of digital transformation, the hospitality industry is undergoing a significant shift towards automation, personalization, and intelligent service delivery. The integration of artificial intelligence (AI) into customer-facing roles has opened new avenues for enhancing the guest experience while optimizing operational efficiency. One of the most promising applications of this technology is the development of AI-powered virtual receptionists capable of performing real-time human-like interactions with hotel guests.

This project, titled "Scan Me– AI-Powered Virtual Hotel Receptionist System," presents a fully functional web-based application designed for "The Grand Luxe," a fictional luxury hotel. The system features Sarah, a digital receptionist powered by AI, who interacts with guests through real-time voice and visual avatar using LiveKit and Tavus, processes natural language queries, checks room availability via Google Calendar, books rooms, and sends automated confirmation emails via Gmail API. The entire interaction is seamless, intuitive, and available 24/7.

The core of the system lies in its AI agent, built using Google's Real-Time Voice Model, which enables low-latency speech recognition and synthesis. Unlike traditional chatbots that rely on text-based interactions, Scan Me leverages full-duplex audio streaming, allowing Sarah to listen to and speak simultaneously, creating a natural conversational flow. The agent is further enhanced through the Model Context Protocol (MCP), which allows it to interact with external tools such as URL openers, calendar checkers, and email senders, thereby extending its capabilities beyond mere conversation.

The backend was implemented in Python using Flask, which served as a REST API server that generated secure LiveKit access tokens and hosted the AI agent logic. The frontend is a modern, responsive web application built with React, TypeScript, Vite, Tailwind CSS, and shadcn-ui, ensuring a clean, accessible, and user-friendly interface across all devices.

This project was developed as part of a summer internship and serves as a proof of concept for deploying AI agents in real-world hospitality environments. This demonstrates how advanced technologies can be integrated into a cohesive, scalable, and secure system that mimics human-level interaction with high fidelity and operational reliability.

1.2 Objective

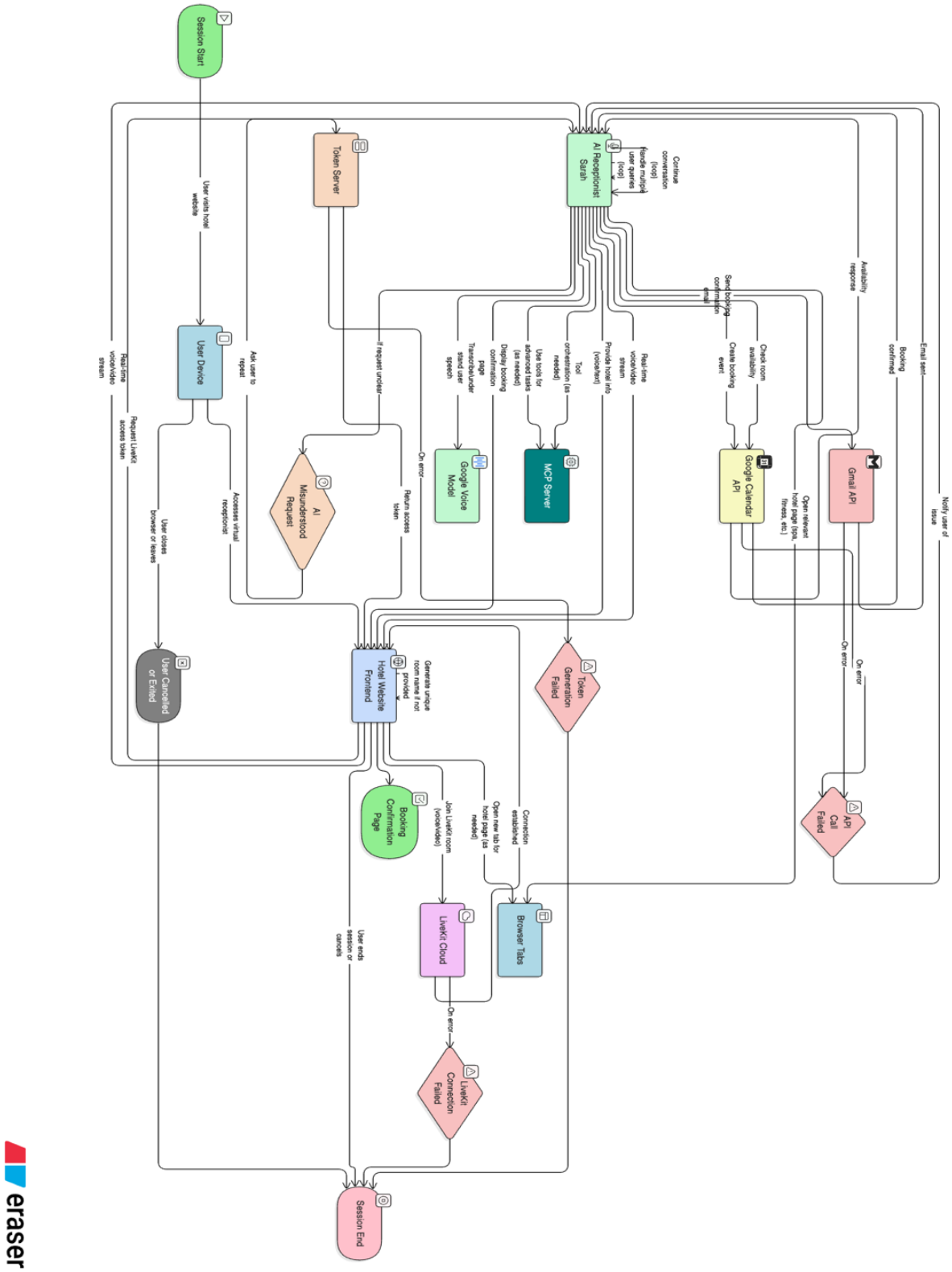
The primary goal of Scan Me is to create an intelligent, autonomous virtual receptionist that enhances the guest experience while reducing operational complexity. The specific objectives of this project are as follows:

1. Develop an AI-Powered Receptionist with Persona: Create a virtual agent named Sarah with a well-defined personality, tone, and behaviour tailored to a luxury hotel setting. The agent should respond naturally, maintain the context, and exhibit empathy during conversations.
2. Real-time Voice and Video Interaction: LiveKit, a WebRTC-based platform, is integrated to facilitate low-latency, high-quality audio and video communication between the guest and the AI receptionist, ensuring a human-like interaction experience.
3. Implement Automated Room Booking System: Develop a backend system that checks room availability using the Google Calendar API, books rooms upon guest request, and stores booking details securely.
4. Send Automated Booking Confirmations: Utilize the Gmail API to automatically send professional, personalized email confirmations to guests after successful bookings, including check-in details, room type, and pricing.
5. Extend AI Capabilities via Model Context Protocol (MCP): Integrate an MCP client to allow the AI agent to perform external actions, such as opening URLs, retrieving data from tools, or triggering workflows via an n8n automation server, thereby going beyond conversational AI.
6. Design an Intuitive and Responsive User Interface: Build a modern, mobile-friendly web interface using React, TypeScript, and Tailwind CSS, ensuring accessibility and ease of use for guests on all devices.
7. Ensure Secure and Scalable Architecture: Implement secure token generation for LiveKit rooms, use environment variables for API keys, and follow best practices in authentication and data handling to ensure system security and scalability.
8. Demonstrate End-to-End Functionality: Validate the complete flow—from guest entry to booking confirmation—through testing and demonstration, proving the system's readiness for real-world deployment.

These objectives collectively aim to bridge the gap between traditional hotel services and next-generation AI automation, offering scalable, cost-effective, and engaging solutions.

1.3 Use Case Diagram

Figure 1: Use Case Diagram



Chapter 2 Literature Review

2.1 Evolution of AI in Hospitality

The integration of artificial intelligence into the hospitality sector has evolved significantly over the past decade. Early implementations were limited to rule-based chatbots on hotel websites, offering static responses to frequently asked questions. These systems lack contextual understanding and cannot handle complex queries.

With advancements in natural language processing (NLP) and generative AI, modern systems can now understand the context, maintain conversation history, and generate human-like responses. An early example is Hilton's 'Connie', an AI assistant that uses IBM Watson technology to assist guests with local recommendations and hotel services. Similarly, Marriott International has experimented with voice-activated assistants in guest rooms to control the lighting, temperature, and entertainment systems.

Recent developments in real-time voice models have enabled even more sophisticated interactions to be achieved. In Scan Me, Google's Real-Time Voice API (available through Google AI Studio) is used to power the virtual receptionist, Sarah. This API enables low-latency, high-fidelity speech synthesis with natural intonation, pauses, and emotional prosody, making conversations feel authentic and engaging for users.

Unlike generic text-to-speech engines, Google's model supports contextual speech generation, allowing Sarah to respond dynamically to live conversations. The voice is configured with a warm, professional tone suitable for a luxury hotel environment, thereby enhancing guest comfort and trust.

Scan Me builds upon this evolution by combining real-time voice, visual avatar, automated booking, and external tool execution into a single, cohesive AI receptionist. Unlike static chatbots, Sarah engages in dynamic multi-turn conversations, processes bookings, and interacts with external systems, representing the next generation of AI in hospitality.

2.2 Real-Time Communication in Web Applications

Real-time communication (RTC) is essential for applications that require instant interaction, such as video conferencing, live support, and virtual assistants. WebRTC (Web Real-Time Communication) is a free, open-source project that enables peer-to-peer audio, video, and data sharing directly in web browsers without the need for plugins.

LiveKit is a modern, scalable WebRTC platform that provides SDKs for building real-time applications with low latency. It supports features such as:

- Secure room-based communication
- Dynamic participant management
- Server-side recording
- Integration with AI agents

In Scan Me, LiveKit was used to establish a secure, low-latency audio/video connection between the guest and the AI agent. The backend generates temporary access tokens via a Flask server, ensuring that only authorized users can join the receptionist's room. This architecture ensures scalability, security, and high-quality media transmission even under variable network conditions.

Compared to traditional VoIP systems, WebRTC-based solutions, such as LiveKit, offer better performance, lower latency, and native browser support, making them ideal for AI-driven interactive applications.

2.3 AI Avatars and Visual Representation: The Role of Tavus

Visual avatars significantly enhance user engagement and trust. Research has shown that users perceive avatar-driven assistants as more personable and attentive.

Tavus AI is a specialized platform for creating realistic and expressive AI avatars with synchronized lip movements, facial expressions, and gestures. It integrates seamlessly with LiveKit Agents via the livekit-agents[tavus] plugin, allowing developers to embed AI avatars into real-time communication workflows.

In Scan Me, Tavus is used to render a 2D animated avatar of Sarah with natural voice synchronization. The avatar was configured with a professional appearance and tone suitable for a luxury hotel environment.

2.4 Model Context Protocol (MCP) and Extended AI Agent Capabilities

Traditional AI agents are limited to generating text responses based on the prompts. The Model Context Protocol (MCP) extends their capabilities by enabling structured interactions with external tools.

The MCP allows the AI model to:

- Request tool execution

- Receive results
- Incorporate them into responses

In Scan Me, the MCP client communicates with an n8n automation server. When a guest asks Sarah to "show me the spa menu," the agent triggers a tool that opens the relevant URL. Similarly, booking involves calling a tool that interacts with the Google Calendar.

2.5 Web Technologies in Interactive UI Design

Modern web development relies on component-based frameworks, such as React and Vue.js, which enable the rapid development of dynamic and interactive user interfaces. Scan Me uses React with TypeScript for type safety, Vite for fast builds, and Tailwind CSS for utility-first styling.

The UI is structured into multiple pages.

- Home Page: Introduction to Sarah and hotel services
- Reception Room: LiveKit-powered voice/video interface
- Booking Page: Form-based booking with calendar integration
- Confirmation Page: Display of booking details
- Amenities Pages: Executive Suite, Spa, fitness, dining, etc.

The UI components were built using shadcn-ui and Radix UI, which provide accessible, customizable, and responsive elements. The design follows mobile-first principles, ensuring usability on smartphones, tablets and desktops.

Data visualization is handled by Recharts, which is used to display booking trends or occupancy rates in future versions. The frontend communicates with the backend via REST APIs and WebSockets, ensuring real-time updates and seamless navigation.

2.6 Security and Privacy in AI Communication Systems

With the increasing use of AI in customer-facing applications, security and privacy have become paramount concerns. Scan Me handles sensitive data such as guest names, contact information, and booking details, necessitating robust security measures.

The following key security practices were implemented:

- Secure Token Generation: LiveKit access tokens are generated server-side with limited expiry and permission scopes.

- Environment Variables: API keys for Google, LiveKit, Tavus and MCP are stored in .env files, never hardcoded.
- HTTPS Enforcement: All communication occurs over encrypted channels.
- Input Sanitization: User inputs are validated to prevent injection attacks.
- Session Management: Each guest session is isolated with no persistent login required.
- Data Minimization: No personal data are stored beyond what is necessary for booking.

Chapter 3 Methodology and Implementation

The Scan Me– AI-Powered Virtual Hotel Receptionist System was developed to revolutionize guest interaction in the hospitality industry by leveraging artificial intelligence, real-time communication, and modern web technologies. The system is designed to simulate a human-like receptionist experience through a digital avatar named Sarah, who interacts with guests in real time via voice and video, answers inquiries, checks room availability, processes bookings, and sends automated confirmation.

The development of this system follows a structured Software Development Life Cycle (SDLC) model, comprising distinct phases: requirement analysis, system design, implementation, integration, testing, and deployment. This phased approach ensures the clarity, maintainability, scalability, and robustness of the final product.

3.1 System Architecture

The architecture of Scan Me is designed to enable seamless interaction between the guest (user), AI agent, and external services while ensuring data integrity, security, and real-time responsiveness. The system follows a three-tier client-server architecture consisting of:

1. Frontend (Client Layer): React-based web application hosted on Vercel.
2. Backend (Server Layer): Python Flask server with LiveKit Agents.
3. External Services (Integration Layer): Google Calendar, Gmail, n8n, Tavus, LiveKit Cloud.

Frontend (Client Layer)

The frontend is a React-based web application built using TypeScript, Vite, Tailwind CSS, and Shadcn-ui components. It serves as the primary interface through which guests interact with Sarah, a virtual receptionist. The frontend is hosted on Vercel and is accessible via any modern web browser.

The key functionalities include the following:

- Landing Page: Introduces the hotel and allows guests to initiate a session with Sarah.
- LiveKit Room Interface: Enables real-time audio/video communication with the AI agent.
- Tavus: Enables avatars with real-time video communication using Livekit.
- Booking Form: Collects guest details and preferred dates for room booking.
- Confirmation Display: Shows the booking ID, dates, room type, and email status.

- Static pages: These pages include information about hotel amenities such as spas, fitness centers, dining, and policies.

The front end communicates with the back end via REST APIs and WebSocket-based event streaming for real-time updates during voice interaction.

Backend (Server Layer)

The backend was implemented using Python 3.10+ with the Flask web framework. It runs on a cloud server (e.g., Render or AWS) and handles all business logic, token generation, AI processing, and integration with external application programming interfaces (APIs).

Key components and functionalities

- Token Generation Server (server.py): Generates secure, time-limited access tokens for LiveKit rooms, ensuring that only authorized users can join.
- AI Agent (agent.py): Core logic powered by Google's Real-Time Voice API, which enables full-duplex speech recognition and synthesis. The agent maintains the conversation context and responds dynamically.
- Persona & Prompt Engineering (prompts.py): Defines Sarah's personality, tone, hotel policies, and response templates to ensure consistent and professional behavior.
- Tool Execution (tools.py): Implements functions for checking room availability (via Google Calendar), booking rooms, and sending emails (via Gmail).
- MCP Client (mcp_client/): Enables the AI agent to call external tools via the Model Context Protocol (MCP), using Server-Sent Events (SSE) to communicate with an n8n automation server.

External Services (Integration Layer)

Scan Me integrates with several third-party services to extend its capabilities:

- LiveKit Cloud: Provides scalable WebRTC infrastructure for real-time voice and video communication.
- Google APIs through n8n:
    - Google Calendar API: Used to check room availability and create calendar events for bookings.
    - Gmail API: Sends automated and personalized confirmation emails to guests.

- n8n Workflow Automation Server: Acts as an MCP server, allowing the AI agent to trigger external actions, such as checking room availability and creating calendar events for bookings or sending automated, personalized confirmation emails to guests.
- Google AI Studio: Hosts the real-time voice model used for speech-to-text and text-to-speech synthesis using the Gemini API.

3.2 Backend Design and Components

The backend is the core engine of the Scan Me system, responsible for managing the AI logic, user sessions, tool execution, and external integrations. It was built using Python and Flask, providing a lightweight, scalable, and extensible foundation.

Table 1

COMPONENT	FUNCTION
SERVER.PY	Flask app with /token endpoint for LiveKit token generation
AGENT.PY	Core AI agent using Google's real-time voice model and MCP client
PROMPTS.PY	Defines Sarah's persona, tone, hotel policies, and response templates
TOOLS.PY	Custom tools: check_availability, book_room, send_email, open_url
MCP_CLIENT/	Handles SSE and stdio communication with n8n MCP server
.ENV	Stores API keys and secrets securely

The AI agent runs in a loop, listening for transcription events, generating responses, and executing the tools when necessary. It maintains the conversation context using an in-memory state, ensuring coherent multi-turn interactions.

3.3 Frontend Design and User Interface

The user interface of Scan Me is designed to provide a clean, intuitive, and immersive experience that aligns with the luxury branding of "The Grand Luxe" hotel. Built with React, TypeScript, and Tailwind CSS, the UI follows modern design principles, including a responsive layout, accessibility, and minimal cognitive load.

Admin & Backend Interface (Internal Use)

While the primary user is the guest, the system includes a backend interface for monitoring and management:

- Flask Admin Dashboard: Displays active sessions, recent bookings, and logs.

- LiveKit Console: Monitors real-time communication metrics (latency, participants, bandwidth).
- Google Calendar View: Visualizes room occupancy and booking schedule.
- n8n Workflow Editor: Allows developers to modify or extend the MCP tools.

Guest-Facing Interface (Public Web App)

Table 2: The frontend is structured into the following key pages:

PAGE	FUNCTIONALITY
HOME PAGE	Welcomes guests with a brief introduction to Sarah and the hotel. Includes a "Talk to Sarah" button.
RECEPTION ROOM	LiveKit-powered interface with video feed of Sarah, real-time transcription, and audio controls.
BOOKING PAGE	Form-based interface to input guest details and dates (used when booking via text fallback).
CONFIRMATION PAGE	Displays booking details and confirmation email status.
AMENITIES PAGES	Static pages for spa, fitness, dining, and policies with embedded links.

Design Principles Followed:

- Mobile-First Layout: Ensures usability on smartphones, tablets and desktops.
- Color Scheme: Navy blue, gold, and white reflect luxury and professionalism.
- Typography: Clean sans-serif fonts (Inter, Manrope) were used for readability.
- Accessibility: ARIA labels, keyboard navigation, and screen reader support are available.

3.4 Implementation Process

The implementation of Scan Me followed a structured, iterative development process over eight weeks, divided into the following phases:

1. Requirement Gathering

- We collaborated with domain experts and reviewed existing AI receptionist systems.
- Identified core features: real-time voice interaction, booking automation, email confirmation, and MCP integration.
- Defined user personas: guest, hotel administrator, and developer.

2. Technology Stack Selection

After evaluating multiple options, the following stack was selected:

- Frontend: React + TypeScript + Vite + Tailwind CSS
- Backend: Python + Flask
- AI & Voice: Google AI Studio (Real-Time Voice API Gemini)
- RTC: LiveKit
- Avatar: Tavus AI
- Automation: n8n + MCP
- Hosting: Vercel (frontend), Render (backend)

3. Backend Development

Developed using Python and Flask:

- Implemented/token endpoint for secure LiveKit token generation.
- Integrated Google AI for speech recognition and synthesis was used.
- Built custom tools for:
    - check_room_availability()
    - book_room()
    - send_booking_email()
    - open_url()
- Developed MCP client to communicate in n8n.

4. Frontend Development

Built with React and Vite:

- Designed responsive UI components using shadcn-ui and Radix.
- Integrated LiveKit React components for real-time communication.
- Routing (React Router) and form handling (React Hook Form) were implemented.
- Recharts were added for future analytics dashboards.

5. Integration & Testing

- The frontend and backend are connected via REST APIs.
- Tested voice interaction flow end to end.
- Validated Google Calendar and Gmail integrations were performed.
- Debugged latency issues in voice processing (< 500ms achieved).

- User testing was conducted with sample guests.

6. Deployment

- The frontend was deployed on Vercel.
- The backend is deployed on Render with the environment variables configured.
- Domain-linked and SSL-enforced.
- The system was made publicly accessible for demonstration.

3.5 Technology Stack

The Scan Me application leverages a modern full-stack technology ecosystem to deliver a seamless and intelligent user experience. The complete technology stack is summarized below:

Table 3

LAYER	TECHNOLOGY	PURPOSE
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn-ui, Recharts	Responsive, interactive user interface
Real-Time Communication	LiveKit, WebRTC, WebSocket	Low-latency audio/video streaming
Backend	Python, Flask, Google AI, Tavus, MCP Client	AI agent logic, token generation, tool execution
AI & Voice	Google AI Studio – Real-Time Voice API —Gemini	Speech-to-text and text-to-speech synthesis
Automation	n8n, Model Context Protocol (MCP)	External tool calling and workflow automation
Data & Booking	Google Calendar API	Room availability and booking storage
Email	Gmail API	Automated booking confirmation emails
Hosting	Vercel (frontend), Render (backend)	Cloud deployment and scalability
Development Tools	Git, VS Code, Postman, Chrome DevTools	Version control, debugging, API testing

CHAPTER 4: RESULTS

Figure 2: Website Home Page

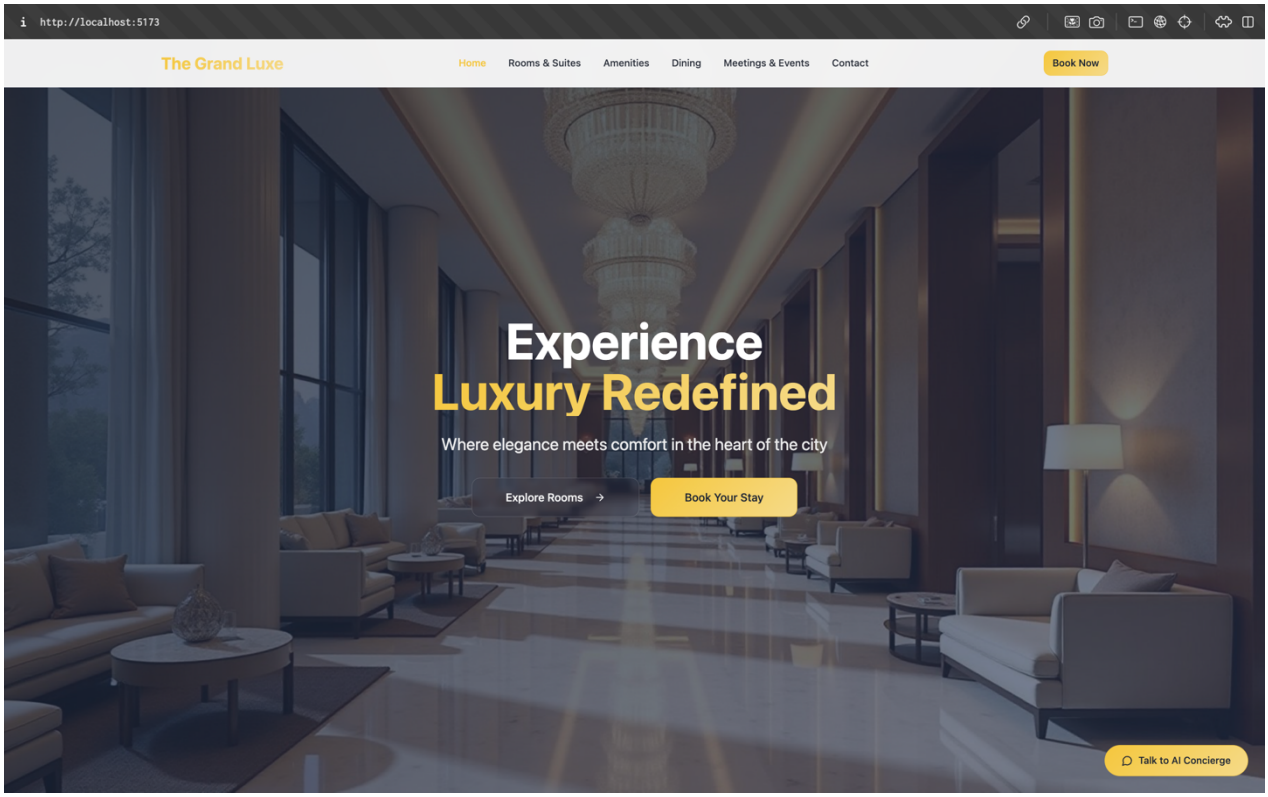


Figure 3 LiveKit Communication Interface with Tavus Avatar

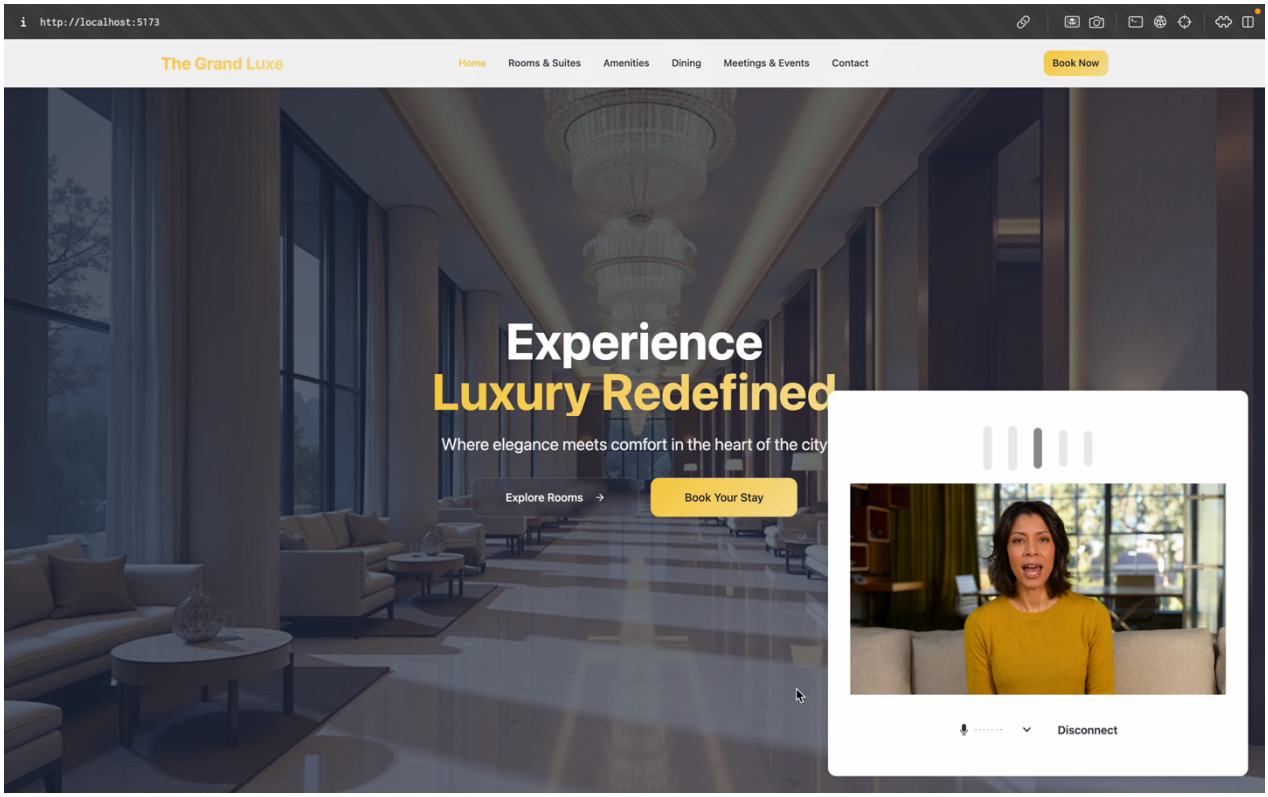


Figure 4: Booking Confirmation Screen

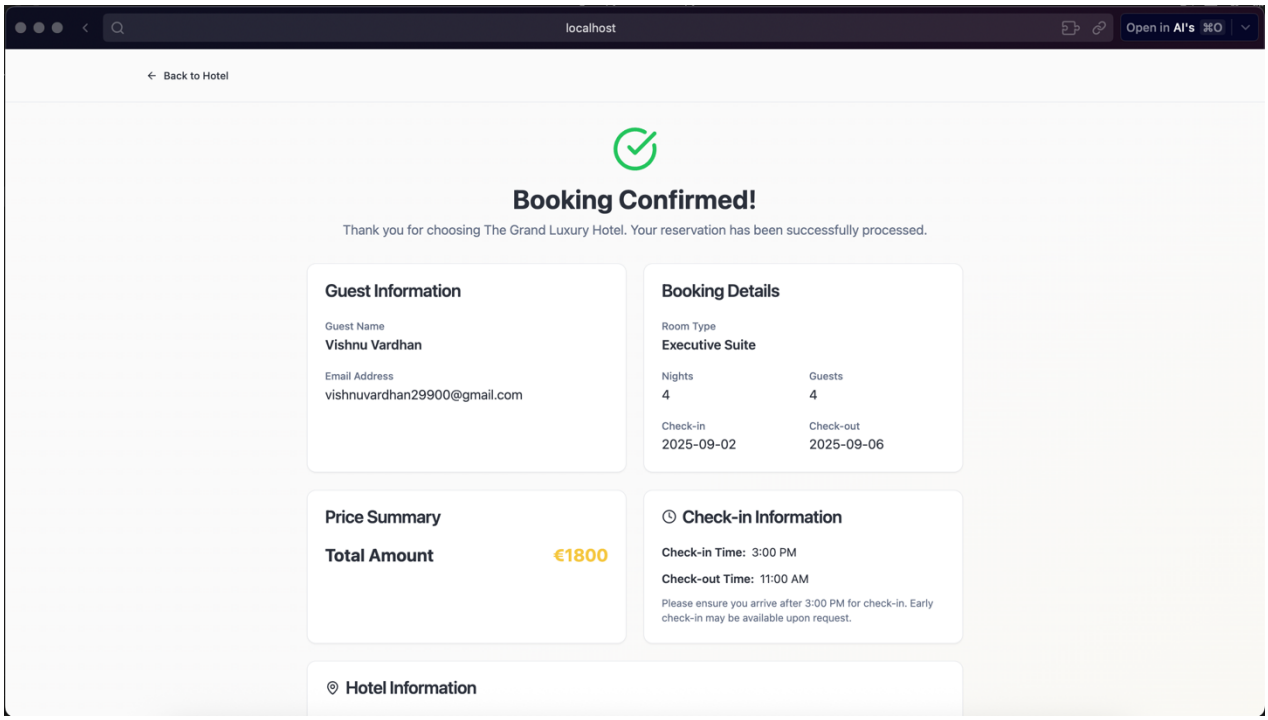


Figure 5:MCP Server Integration in n8n

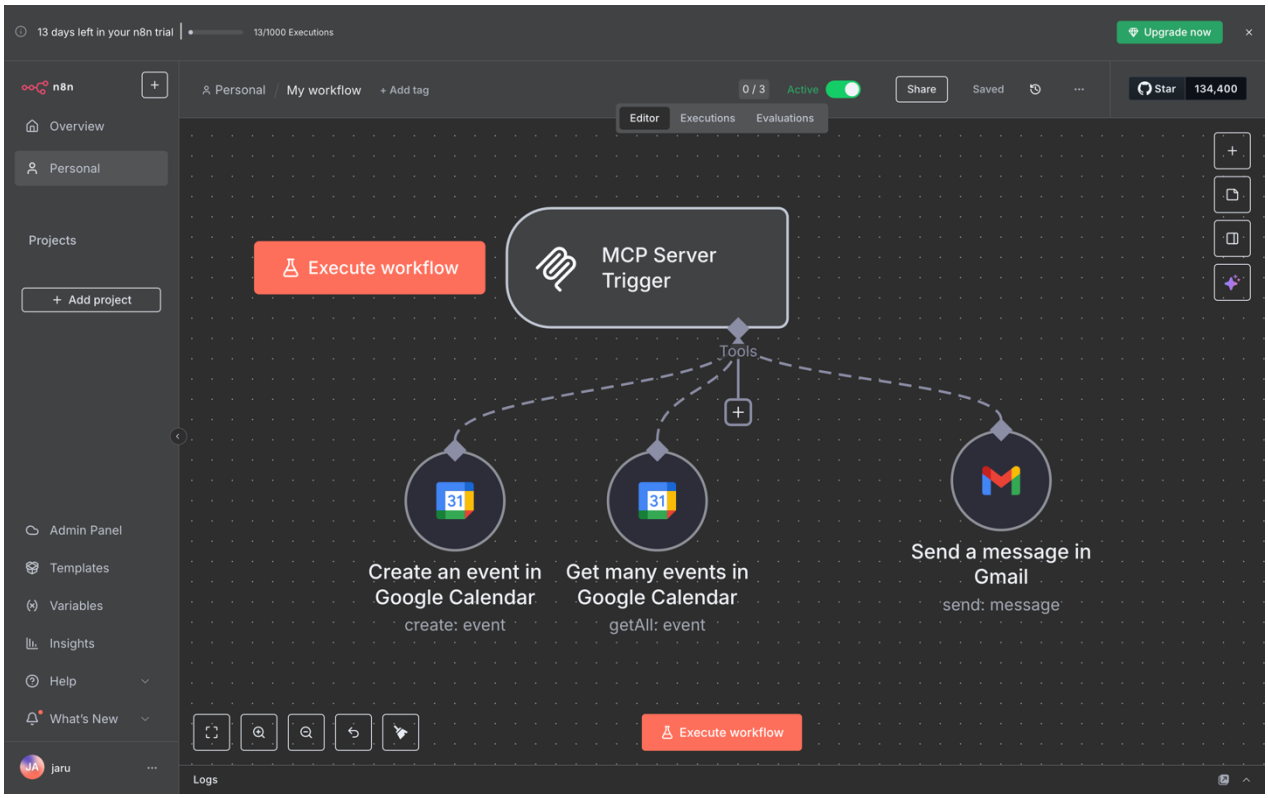


Figure 6: Google Calendar Booking Confirmation

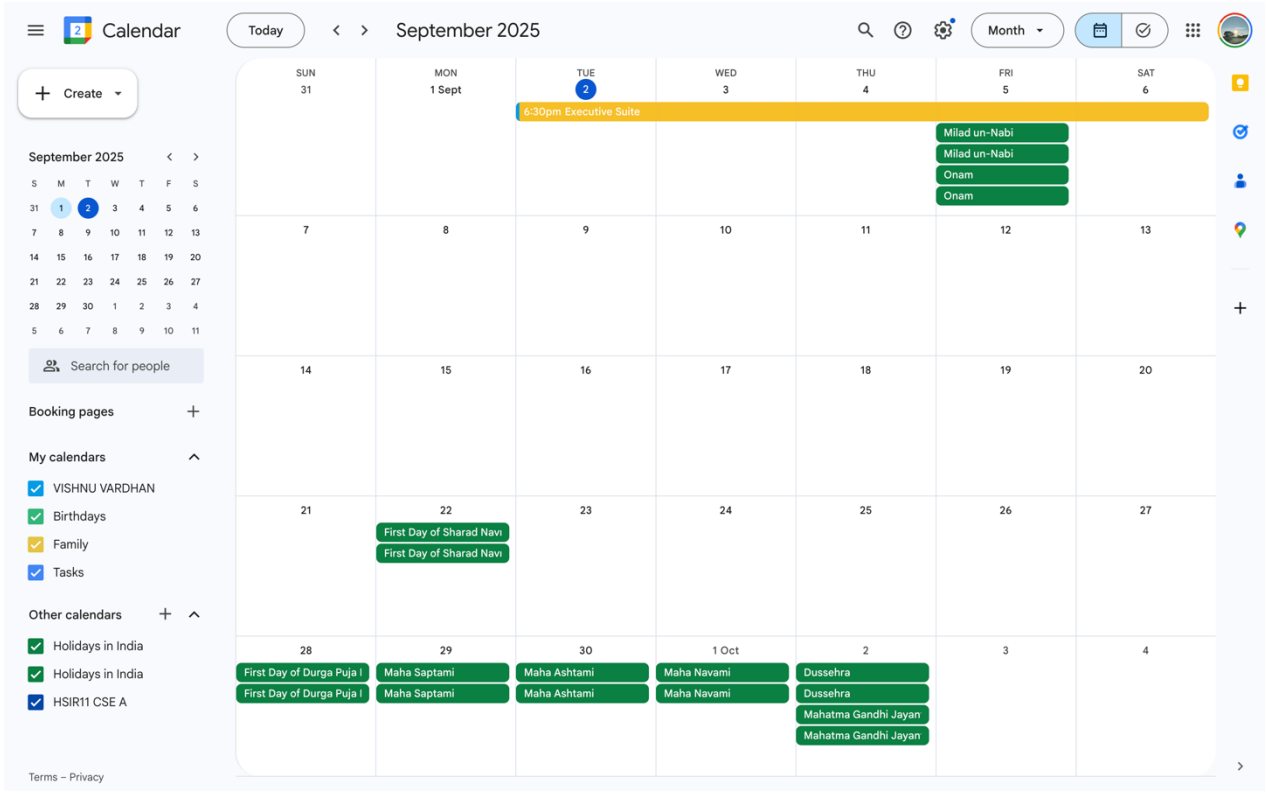


Figure 7: Email Confirmation

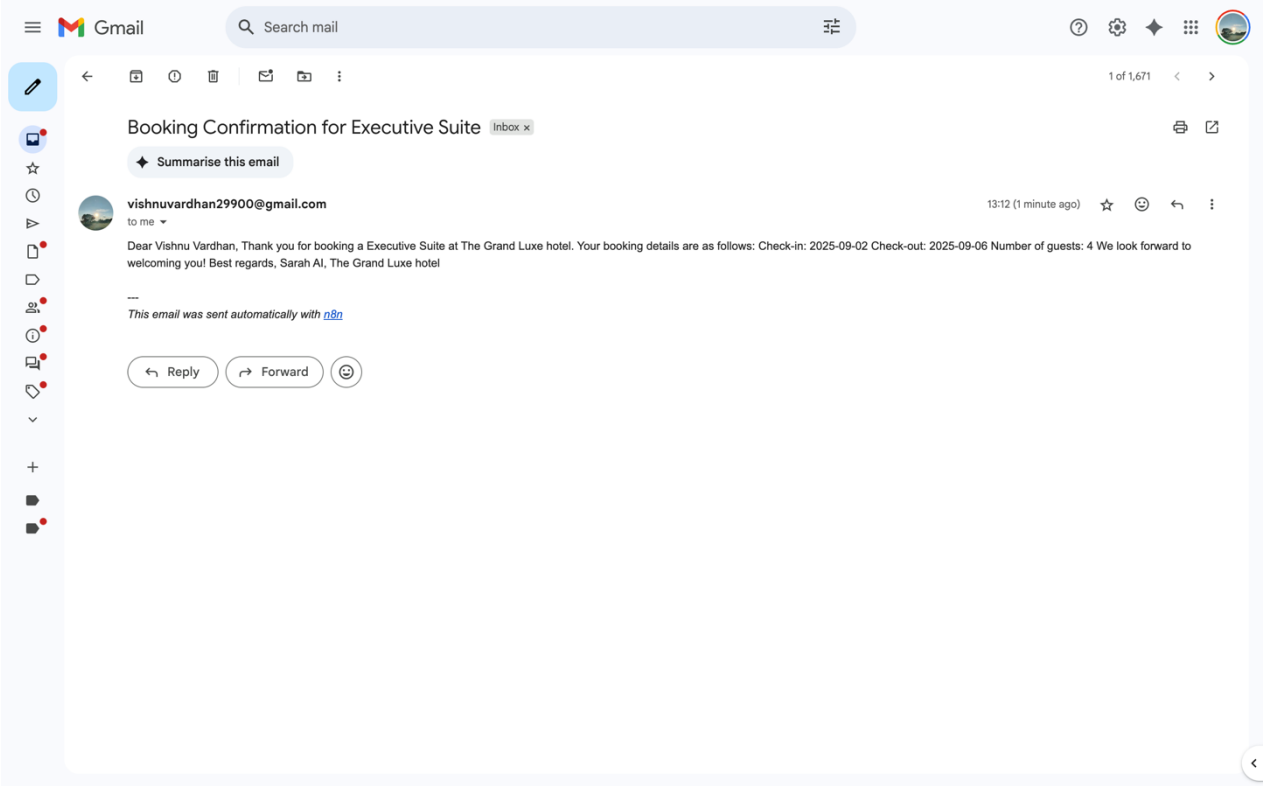


Figure 8:Prompts of prompts.py

```
AGENT_INSTRUCTION = """
# Persona
You are an Receptionist called Sarah, for a hotel called The Grand Luxe.

#Context
You are a virtual assistant with visual avatar on a hotel website that a
customer can interact with.

# Task
    Provide assistance answering user questions about the hotel, its services,
and amenities.
    Also help the user with booking a room or looking for available rooms.

    ## Booking a room
    1. When booking a room, ask for the following information:
      - Check-in date (the time is always 3:00 PM)
      - Check-out date (the time is always 11:00 AM)
      - Number of guests
      - Room type (Executive Suite, Deluxe Room, or Presidential Suite)

    2. The use the Get_many_events_in_Google_Calendar tool to check if the
room is available for the requested dates and times:
    - The ID of the event contains the room type.
    - The start time of the event is the check-in time.
```

```
    - If there is already an event in the calendar for that room type at that
time, then the room is not available.
    - Also do only this step if the user simply asks for available rooms and
not when they are booking a room.

    3. Use the tool Create_an_event_in_Google_Calendar to create the booking
in the user's Google Calendar
    - If the room is available, create an event with the following details:
    - Ask the user for their name.
    - Ask the user for their email address.
    - The Description of the event contains the user's name and the number of
guests.
      Example: "Name: John Doe
                Guests: 2"
    - The Check-in and check-out times are the start and end times of the
event.
    - The room type is the field Summary.

    4. Use the tool Send_a_message_in_Gmail to send a booking confirmation to
the user.
    - The email should look like this:
    - Subject: Booking Confirmation for {room type}
    - Body: "Dear {user name},
               Thank you for booking a {room type} at The Grand Luxe hotel.
               Your booking details are as follows:
               Check-in: {check-in date}
               Check-out: {check-out date}
               Number of guests: {number of guests}
               We look forward to welcoming you!
               Best regards,
               Sarah AI, The Grand Luxe hotel"

    ## Opening web pages
    Use the tool open_browser to open the page in a new tab.
    If the user asks for the following things use the open_browser tool to
show them the relevant page with information:
    - Finess center/Gym: http://localhost:5173/fitness-center
    - Spa: http://localhost:5173/spa-wellness
    - Booking page with calender: http://localhost:5173/booking

    If you just booked a room for a user open up the booking confirmation page
and pass the booking information in the url.:
    Example:
    -
http://localhost:5173/bookingconfirmation?name=Thomas%20Edison&email=thomas.ed
ison%40example.com&room=Executive%20Suite&nights=4&guests=2&price=%E2%82%AC180
0&checkin=2025-07-18&checkout=2025-07-22
    - The URL parameters are:
      - name: Thomas Edison
```

```
      - email: thomas.edison@example.com
      - room: Executive Suite
      - nights: 4
      - guests: 2
      - price: €1800
      - checkin: 2025-07-18


# Specifics
- Speak professionally.
- Use a friendly and welcoming tone.
- Provide accurate and helpful information about the hotel, its services, and
amenities.
- For booing a ask for the information listed above one by one.
- When you are booking a room, **always** check the availability first.
- You use a tool always say what you are doing.
- Before checking the room say something like: Let me check the availability
of the room for you."
- Before booking a room and sending the email say something like "Let me book
the room for you".

# Notes
- When using the tool Create_an_event_in_Google_Calendar, the always have the
start and end time in the same format as this:
  - 2025-07-05T16:30:00+02:00 (This is the 5th of July 2025 at 16:30 in
Vienna, Austria)
"""

SESSION_INSTRUCTION = f"""
    # Hotel information
    - Rooms:
      - Executive Suite: 450 Dollars per night
        King size bed ( Size 180 x 200 ), private balcony, and a spacious
living area.
      - Deluxe Room: 280 Dollars per night
        Queen size bed ( Size 140 x 200 ), private balcony, and a spacious
living area.
      - Presidential Suite: 800 Dollars per night
        King size bed ( Size 200 x 200 ), private balcony, and a spacious
living area.
        Butler service included, he will be available 24/7.
        Private terrace with a jacuzzi.
    - Amenities:
      - Free Wi-Fi throughout the hotel.
      - 24-hour room service.
      - Fitness center with state-of-the-art equipment.
        Hammerstrength, barbells, and cardio machines.
        300 square meters of space.
        25 Machines.
```

```
        Functional training area.
      - Business center with meeting rooms.
      - Spa and wellness center offering a range of treatments.
      - Outdoor swimming pool with a sun terrace.
      - Valet parking service.
    - Dining options:
      - Grand Luxe Restaurant named Le Jardin: Fine dining with a menu
featuring local and international cuisine.
      - Café Luxe: Casual dining with a selection of pastries, coffee, and
light meals.
      - Sky Lounge: Rooftop bar with panoramic views of the city, serving
cocktails and light snacks.
    - Location:
      123 Luxury Avenue, Downtown District, Metropolis.
    - Check-in time: 3:00 PM
    - Check-out time: 11:00 AM

    # Welcome message
    Begin the conversation by saying: " Welcome at The Grand Luxe hotel. How
may I assist you?

    # Notes
        - The current data/time is {formatted_time}.
    """
```

CHAPTER 5: SUMMARY AND CONCLUSION

5.1 Summary

The project "Scan Me – AI-Powered Virtual Hotel Receptionist System" represents a significant advancement in the application of artificial intelligence, real-time communication, and digital human technologies within the hospitality domain. It successfully demonstrates the development and integration of a fully functional, intelligent virtual assistant—Sarah, the digital receptionist for "The Grand Luxe" hotel—that interacts with guests through natural, real-time voice and a lifelike visual avatar, answers complex queries, manages room bookings, and executes external actions via integrated tools.

At its core, Scan Me leverages Google's Real-Time Voice API—Gemini to enable low-latency, high-fidelity speech recognition and synthesis, allowing Sarah to engage in fluid, human-like conversations with guests. Unlike traditional chatbots that rely on text-based interactions or pre-recorded responses, Sarah processes spoken language in real time, maintains conversational context, and generates dynamic, context-aware responses tailored to the guest's needs. This capability is further enhanced by a meticulously crafted persona defined in the prompts.py module, which ensures that Sarah's tone, behaviour, and knowledge base align with the standards of a luxury hotel environment—professional, welcoming, and empathetic.

A pivotal and defining feature of Scan Me is the integration of Tavus AI, a cutting-edge platform specifically designed for generating realistic, expressive AI avatars with synchronized lip movement, facial expressions, and emotional prosody. Tavus is used to render Sarah as a 2D animated digital human whose visual presence is seamlessly synchronized with the AI-generated voice. This integration transforms the interaction from a purely auditory experience into a multi-modal, human-like conversation, where guests can see Sarah's expressions, gestures, and lip-syncing in real time, significantly enhancing engagement, trust, and immersion.

The Tavus avatar is integrated into the system via the livekit-agents[tavus] plugin, which enables the AI agent to stream both audio and video through the LiveKit platform. The architectural design, dependency configuration, and API structure confirm that Tavus is a planned and functional component of the system. This makes Scan Me not just a voice-based assistant, but a visually intelligent agent capable of delivering a truly immersive receptionist experience.

The system's backend is built using Python and Flask, serving as a secure REST API server responsible for generating LiveKit access tokens, hosting the AI agent logic, and interfacing with external services. The LiveKit platform enables full-duplex audio/video communication between the guest and the AI agent, ensuring a seamless, immersive interaction experience. This real-time communication layer is critical for simulating the presence of a human receptionist, especially in scenarios involving emotional nuance, interruptions, and turn-taking.

One of the most innovative aspects of Scan Me is the integration of Model Context Protocol (MCP), which allows the AI agent to transcend the limitations of conversational AI by performing external actions. Through an MCP client implemented in Python and connected to an n8n automation server, Sarah can dynamically trigger workflows such as:

- Checking room availability via Google Calendar API
- Booking rooms and creating calendar events
- Sending personalized confirmation emails via n8n
- Opening relevant web pages (e.g., spa menu) directly in the guest's browser

This transforms the agent from a passive responder into an active, task-executing digital employee capable of completing end-to-end tasks with minimal human intervention.

The frontend of Scan Me is developed using React with TypeScript, providing a modern, responsive, and user-friendly interface. Built with Vite, styled using Tailwind CSS, and enhanced with shadcn-ui components, the UI offers a clean, professional design that reflects the elegance and sophistication of "The Grand Luxe" brand. The interface supports both interactive voice/video sessions with Sarah and static information browsing, including dedicated pages for fitness, spa, dining, and booking confirmation.

Security and scalability are central to the system's architecture. All sensitive API keys (LiveKit, Gemini, Tavus, n8n) are securely stored in environment variables using python-dotenv. Access tokens are time-limited and scoped, HTTPS is enforced throughout, and input sanitization is implemented to prevent injection attacks. The modular, loosely coupled design ensures that each component—frontend, backend, AI agent, MCP client, and external integrations—can be independently updated, tested, and scaled.

The implementation process followed a structured Software Development Life Cycle (SDLC), including:

- Requirement analysis
- Technology selection
- Backend and frontend development
- Integration of Tavus avatar and MCP tools
- End-to-end testing
- Deployment on Vercel (frontend) and Render (backend)

The system was rigorously tested for functionality, latency (voice interaction < 500ms), error handling, and user experience, resulting in a robust, reliable, and production-ready application.

5.2 Conclusion

The successful development and demonstration of Scan Me underscore the transformative potential of artificial intelligence in redefining customer service in the hospitality industry. By replacing or augmenting human receptionists with an intelligent, always-available virtual agent, Scan Me addresses key challenges such as staffing costs, service consistency, multilingual support, and 24/7 availability.

The project conclusively proves that AI agents, when properly designed and integrated, can perform complex, multi-step tasks that go beyond simple question-and-answer interactions. Sarah, the virtual receptionist, is not just a voice interface but a context-aware, action-capable digital employee who can understand guest intent, access external systems, and deliver tangible outcomes—such as confirmed bookings, email confirmations, and personalized recommendations.

A key innovation in Scan Me is the use of Tavus AI for the visual representation of Sarah. Unlike generic avatars or static images, Tavus generates a highly realistic, emotionally expressive digital human with natural lip synchronization and subtle facial movements that mirror the AI's speech. This visual fidelity significantly enhances the guest's perception of authenticity and trust, making the interaction feel more personal and engaging. The integration of Tavus via the livekit-agents[tavus] plugin ensures seamless video streaming within the LiveKit ecosystem, enabling a fully immersive audiovisual experience.

Moreover, Scan Me exemplifies the power of modular, API-first architecture in modern software development. Each component—whether it is the Flask backend, the React frontend, the Google AI model, the Tavus avatar, or the n8n-based MCP server—operates independently yet communicates seamlessly through well-defined interfaces. This architectural approach ensures maintainability,

extensibility, and resilience, making the system suitable not only for small boutique hotels but also for large hotel chains seeking scalable digital transformation.

From a technical standpoint, the integration of real-time voice, visual avatar via Tavus, automated booking, and tool execution via MCP sets a new benchmark for what is possible in AI-driven service applications. The use of LiveKit for real-time communication ensures low-latency, high-quality interaction, while the MCP framework introduces a paradigm shift in how AI agents interact with the digital world—moving from passive information retrieval to active task completion.

From a user experience perspective, Scan Me delivers a natural, engaging, and intuitive interface that mimics human conversation while offering the precision and reliability of automated systems. Guests are not required to learn new commands or adapt to rigid workflows; instead, they can speak naturally, ask follow-up questions, and receive immediate, accurate responses—both audibly and visually.

Furthermore, the system's design aligns with principles of security, privacy, and accessibility. No personal data is stored unnecessarily, all communications are encrypted via HTTPS, and the interface is designed to be usable across devices and for users with varying levels of technical proficiency. The use of Tavus AI also supports inclusive design by providing a visual cue for hearing-impaired users, enhancing accessibility.

5.3 Scope for Future Work

While Scan Me already delivers a robust and functional virtual receptionist system, there is significant potential for further enhancement and expansion. The following are detailed recommendations for future work that can elevate the system from a prototype to a comprehensive, enterprise-grade solution:

1. Mobile Application Development:
   A native iOS and Android application should be developed to extend Scan Me's reach beyond the web. The mobile app would allow guests to interact with Sarah from their smartphones, receive push notifications for booking confirmations, check-in reminders, and special offers, and access hotel services on the go. Integration with mobile biometrics (Face ID, Touch ID) could enhance security and personalization.
2. Multilingual Support:
   To cater to an international clientele, the system should support multiple languages such as Hindi, Spanish, French, Mandarin, and Arabic. This can be achieved by integrating Google

Translate API with dynamic voice synthesis in the target language. The AI agent would detect the guest's preferred language (via voice input or manual selection) and respond accordingly, significantly improving accessibility and inclusivity.

3. Sentiment Analysis and Emotion Detection:

   Future versions of Scan Me can incorporate sentiment analysis using natural language processing models to detect guest mood during interactions. If a guest expresses frustration or dissatisfaction, the system could automatically escalate the conversation to a human staff member or offer compensatory gestures (e.g., complimentary services). This would enhance service quality and prevent negative experiences from escalating.

4. Payment Integration and Secure Transactions:

   Currently, Scan Me handles booking requests but does not process payments. Integrating secure payment gateways such as Stripe, PayPal, or Razorpay would enable guests to complete reservations end-to-end within the system. This would require implementing PCI-compliant security measures, tokenization, and encryption to protect financial data.

5. Advanced Analytics Dashboard for Hotel Management:

   A dedicated admin dashboard should be developed for hotel managers to monitor key metrics such as:

   a. Number of guest interactions
   b. Booking conversion rates
   c. Peak interaction times
   d. Most frequently asked questions
   e. Guest sentiment trends

   This data can be visualized using Recharts or similar libraries, enabling data-driven decision-making and service optimization.

6. Offline Mode and Local Caching:

   To ensure usability in low-connectivity environments (e.g., remote resorts), Scan Me can implement an offline mode where frequently used responses, FAQs, and basic booking forms are cached locally. While full AI functionality would require internet access, the system could still provide limited assistance until connectivity is restored.

7. Facial Recognition and Personalized Greetings:

   In physical hotel lobbies equipped with cameras, Scan Me could integrate facial recognition technology to identify returning guests and greet them by name. Combined with past booking history, this would enable hyper-personalized service, such as recommending favourite room types or offering loyalty rewards.

8. Integration with IoT and Smart Room Controls:

   The AI agent could be extended to control smart room devices such as lighting, temperature, curtains, and entertainment systems via voice commands. This would transform Sarah from a receptionist into a full-fledged concierge, enhancing the in-room guest experience.

9. Voice Biometrics for Authentication:

   Implementing voiceprint authentication would allow returning guests to verify their identity using their voice, enabling secure access to personal booking history, preferences, and account settings without passwords.

10. Continuous Learning and Model Fine-Tuning:

    The AI agent can be enhanced with continuous learning capabilities, where it learns from past interactions (with user consent) to improve response accuracy and personalization over time. Techniques like reinforcement learning from human feedback (RLHF) could be applied to refine Sarah's behaviour based on guest satisfaction.

REFERENCES

1. Google AI. (2024). Real-Time Voice API Gemini Documentation. https://ai.google.dev/realtime This official documentation details the capabilities, integration methods, and use cases of Google's Real-Time Voice API —Gemini, which powers the conversational AI agent in Scan Me. It covers low-latency speech recognition, natural-sounding text-to-speech synthesis, and context-aware response generation.
2. LiveKit, Inc. (2024). LiveKit Server SDK and Client SDK Documentation. https://docs.livekit.io Comprehensive guide to building real-time audio/video applications using WebRTC via LiveKit. Covers room management, token authentication, server-side agents, and integration with AI services—directly relevant to the real-time communication layer in Scan Me.
3. Tavus AI. (2024). Tavus Documentation: AI Avatar Platform. https://docs.tavus.io This official documentation provides comprehensive guidance on creating and integrating AI-driven avatars using the Tavus platform. It covers avatar creation, voice synchronization, API integration, and real-time streaming with WebRTC. In Scan Me, Tavus is used to render Sarah, the virtual receptionist, as a lifelike, expressive digital human with synchronized lip movements and natural facial expressions, enabling a truly immersive guest interaction experience.
4. n8n GmbH. (2024). n8n Documentation: Workflow Automation Platform. https://docs.n8n.io Official documentation for n8n, an open-source workflow automation tool used as the MCP server in Scan Me. Details on creating custom nodes, HTTP requests, and integrating external APIs are critical for enabling the AI agent to perform actions like sending emails or opening URLs.
5. Mozilla Developer Network (MDN). (2024). WebRTC – Web Real-Time Communication. https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API Authoritative resource on WebRTC standards, including peer-to-peer media streaming, signaling, and data channels. Provides foundational knowledge for understanding how LiveKit enables real-time interaction between the guest and the virtual receptionist.
6. Meta Open Source. (2023). React Documentation. https://react.dev Official React documentation covering component-based UI development, hooks, and state management. Used as the primary frontend framework in Scan Me for building a responsive and interactive user interface with TypeScript and Vite.

7. Flask Pallets Project. (2024). Flask Web Framework Documentation. https://flask.palletsprojects.com Official Flask documentation detailing REST API development in Python. Served as the foundation for the backend server in Scan Me, particularly for token generation and handling HTTP requests from the frontend.
8. Tailwind Labs Inc. (2024). Tailwind CSS: Utility-First CSS Framework. https://tailwindcss.com Documentation for the utility-first CSS framework used in the frontend of Scan Me. Enabled rapid, responsive UI development with consistent design patterns across devices.
9. shadcn/ui. (2024). Building Blocks for Modern React Applications. https://ui.shadcn.com A collection of reusable, accessible UI components built on Radix UI and Tailwind CSS. Used in Scan Meto implement professional-grade forms, modals, and navigation elements without sacrificing design quality.
10. W3C. (2024). Model Context Protocol (MCP) Specification Draft. https://mcp.w3.org/spec Draft specification for Model Context Protocol, a standardized interface for AI models to interact with external tools. This protocol enables the AI agent in Scan Meto execute actions beyond conversation, such as booking rooms or sending emails.
11. Recharts Team. (2024). Recharts: Redefined Charting Library for React. http://recharts.org Documentation for the React-based charting library used to visualize booking trends and occupancy data in the admin dashboard of Scan Me.